AI Lab # 5

This lab explores informed search strategies by implementing Greedy Best-First Search and A* Search.

Artificial Intelligence (LAB)

Instructor: Mubbashir Ahmed

Email: mubbashir.ahmed@iqra.edu.pk

Date: 09 August, 2026

Today's Agenda

IQRA UNIVERSITY IU

01 What is Informed Search?

02 What is Greedy Best-First Search?

03 Algorithm Steps and Example

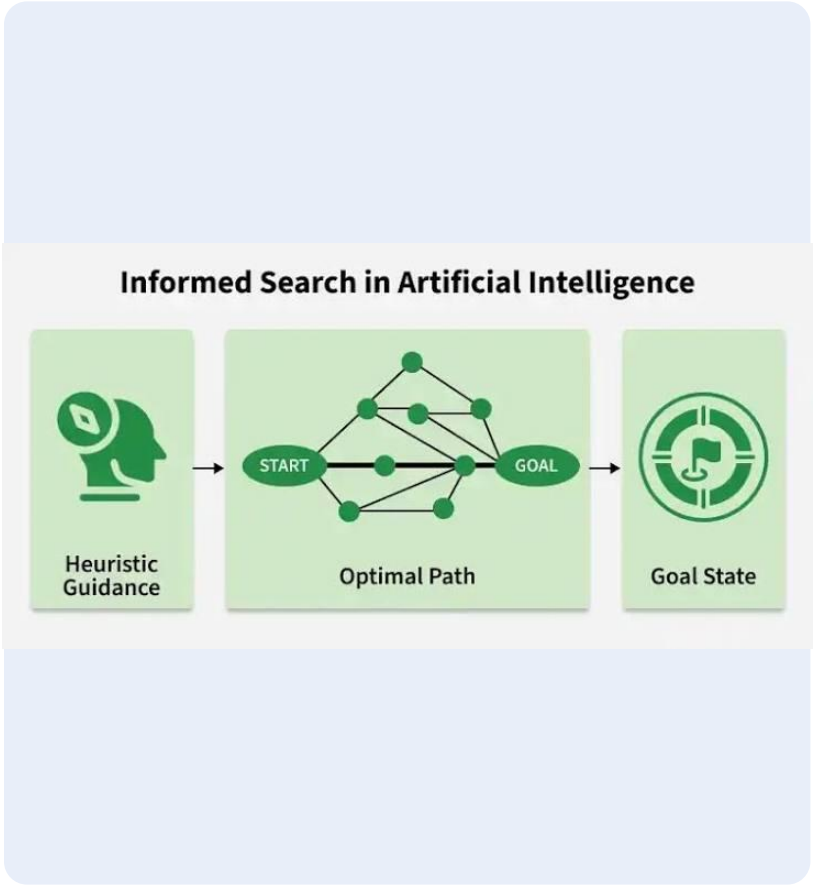
04 What is A* Search?

05 Lab Tasks

Informed Search

- Informed search (also called ***heuristic search***) uses **extra information** (called a ***heuristic***) to find solutions faster and smarter than uninformed methods like BFS, DFS, or UCS.
- A heuristic is like a guess or clue that tells the algorithm how close you are to the goal.

Imagine you're playing hide and seek, and someone tells you: "I'm here".

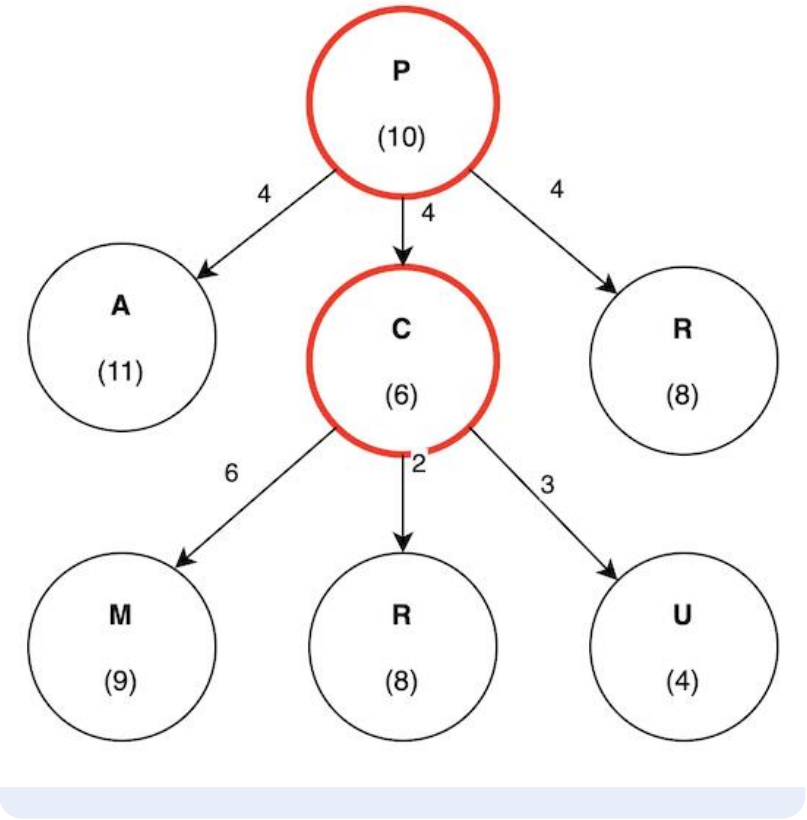


What is Greedy Best-First Search?

- Greedy Search always picks the node that looks closest to the goal, using the heuristic only.
- It does not care about how far you've already come, it only cares about what seems best right now

Applications:

- GPS Navigation – to quickly reach a destination using shortest-looking turns.
- Job scheduling – picking the task that looks quickest to finish.



Priority queue *is a special kind of list where items with the smallest number comes out first*

Heuristic Formula For Greedy Best-First Search

$$f(n) \geq h(n)$$

- Where,
- h(n) = your guessed distance from node n to the goal node.
- f(n) = the actual shortest distance from node n to the goal node.

Manhattan Distance

When we have points and want to know how far they are from each other, or measure their similarity or dissimilarity, we use a Manhattan distance formula.

Manhattan Distance = $\Sigma_{i=1}^{n} |x_i - y_i|$

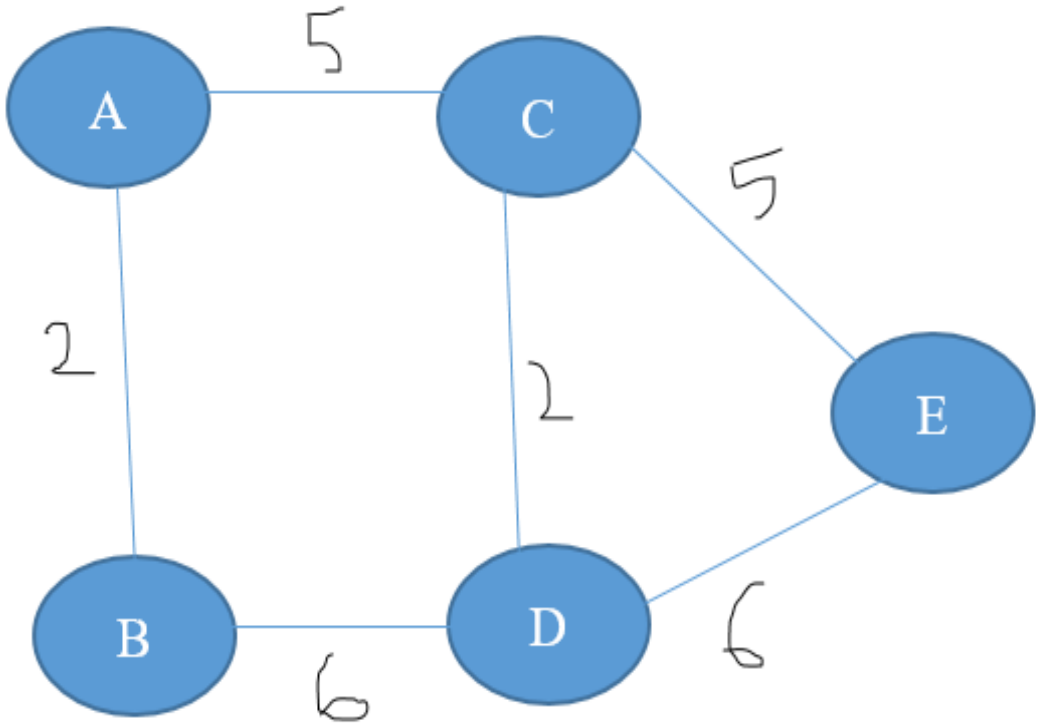
Example:

X1 =(2,5), Y1 =(3,4)

|2-3| + |5-4| = 2

Algorithm Steps

1. **Add the start node:** Put the starting node into a priority queue, ranked by heuristic value h(n)
2. **Pop the best node:** Take the node with the lowest h(n) value from the queue and mark it as visited
3. **Check for goal:** If this node is the goal, stop and return the path
4. **Push unvisited neighbors:** Add all unvisited neighbor nodes to the priority queue, each ranked by their own h(n) value
5. **Repeat until the goal is found or queue is empty:** Keep repeating step 2 to 4 until the goal is found or the priority queue is empty



heuristic/guesses

A -> 10

B -> 8

C -> 5

D -> 3

E -> 0

Start node = A

Goal node = E

Example of Greedy Best-First Search

Now that we've covered Greedy Best-First Search, let’s move towards practical code-based example

Greedy Best-First Search

Greedy Best-First Search on nodes A, B, C, D, E. Starting node is A and Goal node is E.

```
import heapq
graph = { 'A': ['B', 'C'], 'B': ['D'], 'C': ['E'], 'D': ['E'], 'E': [] }
heuristic = { 'A': 10, 'B': 8, 'C': 5, 'D': 3, 'E': 0 }

def greedy_search(start, goal):
    pq = [(heuristic[start], start, [start])]
    visited = set()
    while pq:
        h, current, path = heapq.heappop(pq)
        if current == goal:
            return path
        if current not in visited:
            visited.add(current)
            for neighbor in graph[current]:
                if neighbor not in visited:
                   heapq.heappush(pq, (heuristic[neighbor], neighbor, path + [neighbor]))
    return None

result = greedy_search('A', 'E')
print("Greedy Best First Search:", result)
```

What is A* Search?

- A* is a smart way to find the shortest and best path from one place to another.
- It’s smarter than greedy search because it doesn’t just look at how close the goal is but also remembers how far we’ve already come.

Imagine driving to a mall using two routes:

- **Route 1 looks very close to the mall on the map (low h(n)) but has terrible traffic (high g(n) once you're actually on it)**
- **Route 2 looks a bit farther on the map but has no traffic**

Greedy would blindly pick Route 1 because it *looks* closest. A* checks both the traffic already experienced and the remaining distance, so it correctly picks the actually-faster route.

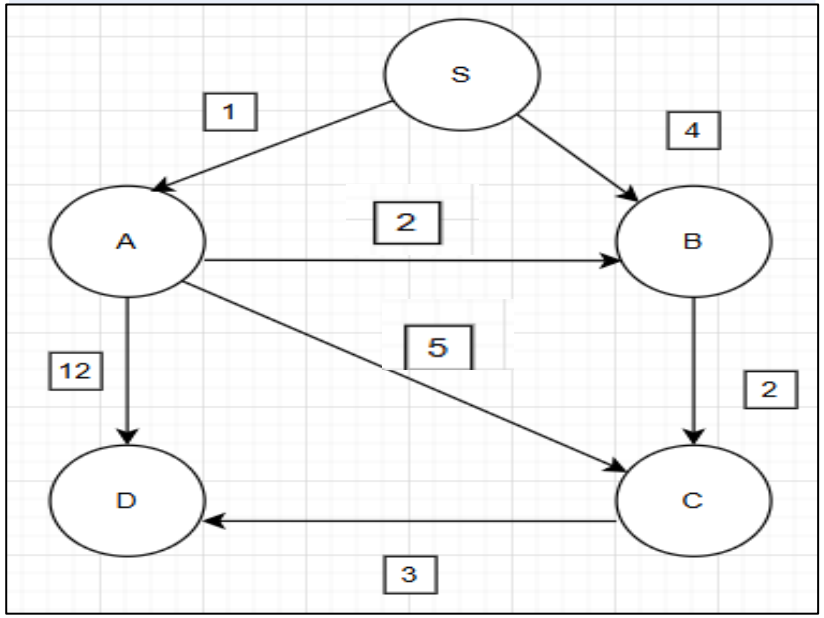
Heuristic Formula for A* Search

$$f(n) = g(n) + h(n)$$

Where,

g(n) = cost to reach from one node to another

h(n): heuristic of the nodes to reach goal



Algorithm Steps

1. **Add the start node:** Put the starting node into a priority queue, with f(n) = g(n) + h(n), where g(start) = 0

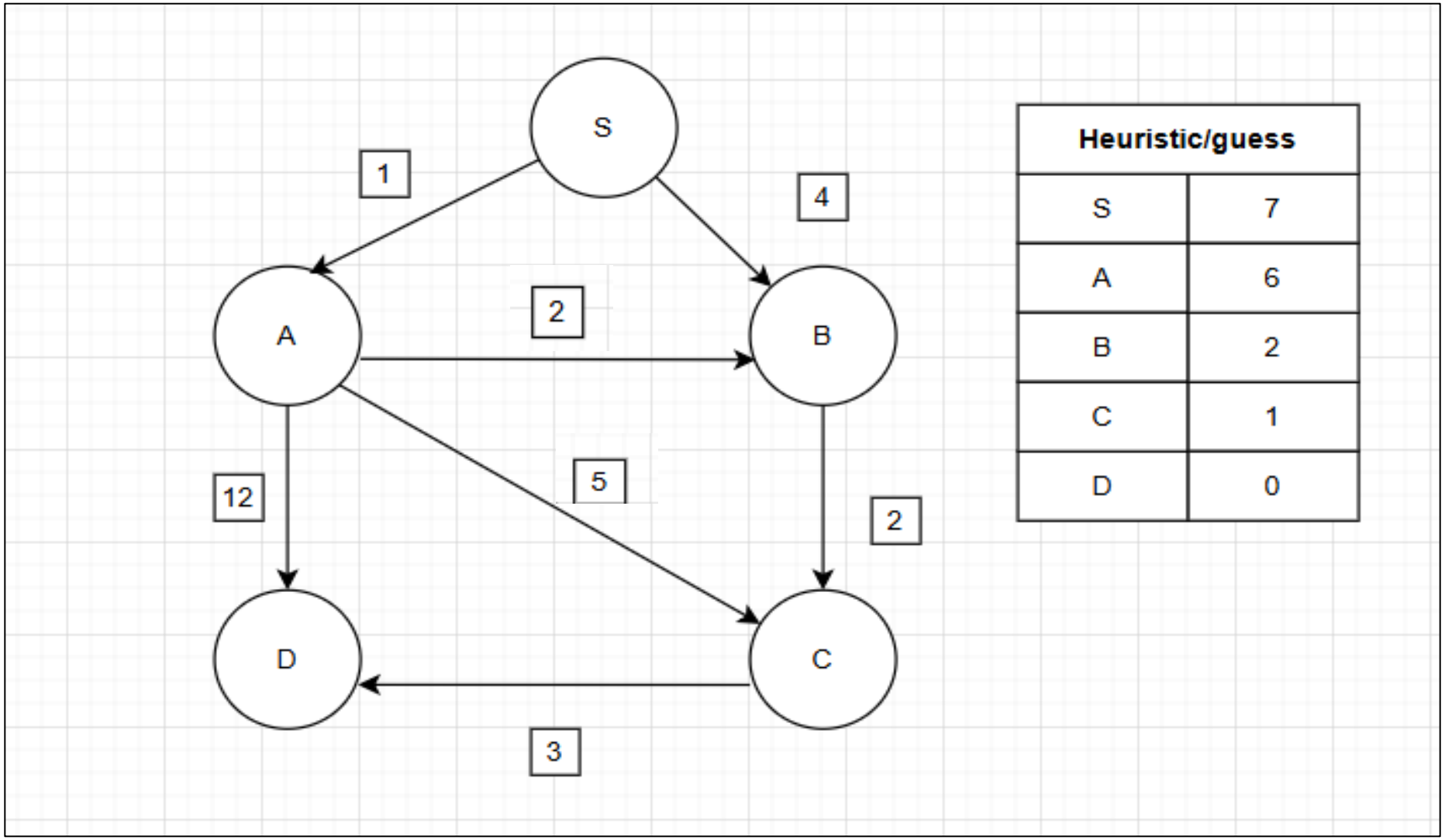
2. **Pop the best node:** Take the node with the lowest f(n) value from the queue

3. **Check for goal:** If this node is the goal, stop and return the path

4. **Calculate costs for neighbors:** For each unvisited neighbor, compute:
   new_g = g(current) + cost to reach neighbor; new_h = heuristic(neighbor); new_f = new_g + new_h

5. **Push updated neighbors:** Add each neighbor to the priority queue with its new_f value

6. **Repeat until the goal is found or queue is empty:** Keep repeating step 2 to 5 until the goal is found or the priority queue is empty



Heuristic/guess	
S	7
A	6
B	2
C	1
D	0

IQRA UNIVERSITY IU

Example of A* Search

Now that we've covered A* Search, let’s move towards practical code-based example

A* Search

A* Search on nodes A, B, C, D, E. Starting node is A and Goal node is E.

```
import heapq
GRAPH = { 'A': [('B', 2), ('C', 5)], 'B': [('D', 3)], 'C': [('E', 5)], 'D': [('E', 2)], 'E': [] }
HEURISTICS = { 'A': 10, 'B': 8, 'C': 5, 'D': 3, 'E’: 0 }

def a_star(start, goal):
    g_start = 0
    h_start = HEURISTICS[start]
    f_start = g_start + h_start
    queue = [(f_start, start, [start], g_start)]
    visited = []
    while queue:
        f_cost, node, path, g_cost = heapq.heappop(queue)
        if node in visited:
            continue
        if node == goal:
            print("\nGoal Reached!")
            print(" -> ".join(path), "=", g_cost)
            return path
        visited.append(node)
        for child, cost in GRAPH[node]:
            if child in visited:
                continue
            new_g = g_cost + cost
            new_h = HEURISTICS[child]
            new_f = new_g + new_h
            new_path = path + [child]
            heapq.heappush(queue, (new_f, child, new_path, new_g))
    return None

a_star('A', 'E')
```

ACTIVITY

Lab Task: Greedy Best-First Search (Informed Search)

Graph:

```
graph = {
    'S': ['A', 'B'],
    'A': ['C', 'D'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': ['F'],
    'E': ['F'],
    'F': []
}
```

Heuristic values (estimated distance to goal F):

```
heuristic = {
    'S': 12,
    'A': 9,
    'B': 8,
    'C': 6,
    'D': 4,
    'E': 5,
    'F': 0
}
```

Implement GBFS using a priority queue (heapq) that always expands the node with the lowest heuristic value. Start at node 'S' and Goal node is 'F'.

- ***Print the final path found***
- ***Print the heuristic value at each step chosen (show the trace, not just the final answer)***

Thank You

Questions or discussions are welcomed!